A MAJOR PROJECT REPORT ON

FinRAG AI - A Unified AI Framework for Financial Risk and Retrieval-Augmented Intelligence

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering (Artificial Intelligence)

By

TALABATTULA JITENDRA SRI -- 23A81A4361

S.V.M.S. VISHAL -- 23A81A4357

G. USHA -- 23A81A4314

M. CHARAN KRISHNA VARMA -- 23A81A4334

Under the Esteemed Supervision of

Mr. K. KALYAN

Assistant Professor, Department of AI & ML

Department of Computer Science and Engineering (Artificial Intelligence)

SRI VASAVI ENGINEERING COLLEGE (Autonomous)

(Affiliated to JNTUK, Kakinada)

Pedatadepalli, Tadepalligudem-534101, A.P

2026-27

SRI VASAVI ENGINEERING COLLEGE (Autonomous)

Department of Computer Science and Engineering (Artificial Intelligence)

Pedatadepalli, Tadepalligudem -- 534101, A.P

CERTIFICATE

This is to certify that the Project Report entitled

"FinRAG AI - A Unified AI Framework for Financial Risk and Retrieval-Augmented Intelligence"

submitted by

TALABATTULA JITENDRA SRI -- 23A81A4361

S.V.M.S. VISHAL -- 23A81A4357

G. USHA -- 23A81A4314

M. CHARAN KRISHNA VARMA -- 23A81A4334

in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence) from Sri Vasavi Engineering College (Autonomous), affiliated to JNTUK, Kakinada, is a record of bonafide work carried out by them under our supervision and guidance during the academic year 2026-27.

This project report has not been submitted to any other University or Institution for the award of any degree or diploma.

Project Guide	Head of the Department
Mr. K. KALYAN	Dr. G. Loshma

Assistant Professor, AI & ML

Professor & Head, Dept. of AI & ML

External Examiner

DECLARATION

We hereby declare that the project report entitled "FinRAG AI - A Unified AI Framework for Financial Risk and Retrieval-Augmented Intelligence" submitted by us in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence) to Sri Vasavi Engineering College (Autonomous), Tadepalligudem, affiliated to JNTUK, Kakinada, is a genuine work carried out by us.

The work presented in this report has not been submitted previously by us or by any other person for the award of any degree, diploma, associateship, fellowship, or any other similar title or prize to any Institute, University, or Board. We have faithfully acknowledged all sources of information used in this work. We take full responsibility for any errors or omissions.

Project Associates:

TALABATTULA JITENDRA SRI -- 23A81A4361

S.V.M.S. VISHAL -- 23A81A4357

G. USHA -- 23A81A4314

M. CHARAN KRISHNA VARMA -- 23A81A4334

ACKNOWLEDGEMENT

First and foremost, we sincerely salute our esteemed institute Sri Vasavi Engineering College (Autonomous), Tadepalligudem, for providing us this golden opportunity to undertake this major project and nurturing an environment that encourages innovation and applied research.

Our sincere gratitude goes to our project guide, Mr. K. KALYAN, Assistant Professor, Department of Artificial Intelligence and Machine Learning, for his invaluable guidance, continuous encouragement, and insightful suggestions throughout the development of FinRAG AI. His expertise in machine learning and financial systems was instrumental in shaping the direction of this project.

We express our sincere thanks and heartful gratitude to Dr. G. Loshma, Professor and Head of the Department of Artificial Intelligence and Machine Learning, for providing the necessary infrastructure, support, and motivation required for the successful completion of this project.

We express our sincere thanks and heartful gratitude to Dr. Ch. Rambabu, Principal, Sri Vasavi Engineering College (Autonomous), for providing a favourable academic environment and extending all the required facilities.

Our special thanks to the management and all the teaching and non-teaching staff members of the Department of Artificial Intelligence and Machine Learning for their guidance and support throughout our academic journey.

We would like to express our gratitude to our parents and friends who extended their moral support and helped us to complete this project successfully.

Project Associates:

TALABATTULA JITENDRA SRI -- 23A81A4361

S.V.M.S. VISHAL -- 23A81A4357

G. USHA -- 23A81A4314

M. CHARAN KRISHNA VARMA -- 23A81A4334

ABSTRACT

FinRAG AI is an AI-powered financial intelligence platform that integrates two complementary intelligence engines within a unified web-based interface. The Transaction Intelligence Engine employs a hybrid machine learning pipeline combining a RandomForestClassifier (200 estimators, class-balanced, depth-limited) with an IsolationForest anomaly detector trained exclusively on legitimate transactions. Transaction features -- amount, channel, account age, and KYC verification status -- are used to generate a hybrid risk score (75% fraud probability, 25% anomaly score). Alerts are ranked by severity and presented with human-readable explanations of the contributing risk factors.

The Financial RAG Engine processes uploaded financial documents (PDF, HTML, Markdown, and plain text) through a pipeline of structure-preserving text extraction, section-aware chunking, LLM-generated metadata enrichment, and dual-mode retrieval combining TF-IDF/BM25 sparse retrieval with dense vector retrieval using Google Gemini embeddings (gemini-embedding-001, 768 dimensions). Candidate passages are ranked via Reciprocal Rank Fusion, and a Google Gemini generative model (gemini-flash-lite-latest) produces grounded answers with evidence citations. Retrieval is evaluated using the FinDER benchmark; dense retrieval achieves Recall@10 of 88% and MRR of 0.778 on a 25-query evaluation over 630 annotated passages.

The platform is built on Flask 3.x with MongoDB for data persistence, PyJWT + bcrypt + OTP for authentication, and Chart.js-powered dashboards for transaction analytics. Ten functional modules are delivered: authentication, transaction import and management, fraud alert review, spending analytics, predictive forecasting, credit health scoring, investment portfolio tracking, loan EMI calculation, AI model insights, and the FinRAG Copilot for document question answering.

TABLE OF CONTENTS

Preliminary Pages

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

List of Figures viii

List of Abbreviations ix

Chapter 1 -- Introduction 1

1.1 Introduction 1

1.2 Background 2

1.3 Problem Statement 3

1.4 Motivation 4

1.5 Objectives 4

1.6 Scope of the Project 5

1.7 Proposed Solution 5

1.8 Organization of the Report 6

Chapter 2 -- Literature Survey 7

Chapter 3 -- System Study and Requirements Analysis 13

Chapter 4 -- System Design and Architecture 20

Chapter 5 -- Dataset and Data Processing 30

Chapter 6 -- Technologies and Methodology 36

Chapter 7 -- Implementation and Module Description 44

Chapter 8 -- RAG Pipeline and Transaction Intelligence 55

Chapter 9 -- Testing and Performance Evaluation 64

Chapter 10 -- Results and Website Screenshots 73

Chapter 11 -- Conclusion and Future Scope 79

References 83

Appendix A -- API Endpoint Reference **86**

Appendix B -- Sample Transaction CSV Format **89**

Appendix C -- Setup and Installation Instructions **90**

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Overall System Architecture	21
4.2	Data Flow Diagram	23
4.3	RAG Pipeline Architecture	25
4.4	MongoDB Database Schema	27
8.1	RAG Pipeline Workflow	56
8.2	Transaction Risk Scoring Pipeline	62
10.1	Login / Authentication Page	73
10.2	Main Dashboard	74
10.3	Upload Transaction Data Interface	75
10.4	Fraud Alerts with Severity	75

	Levels	
10.5	FinRAG Copilot -- Question Answering Interface	76
10.6	Loan EMI Calculator	77
10.7	Credit Health Dashboard	77
10.8	Investment Portfolio Interface	78

LIST OF ABBREVIATIONS

Abbreviation	Full Form

AI	Artificial Intelligence
API	Application Programming Interface
BFSI	Banking, Financial Services, and Insurance
BM25	Best Match 25 (probabilistic retrieval function)
CSE	Computer Science and Engineering
CSV	Comma-Separated Values
EMI	Equated Monthly Instalment
FinDER	Financial Document Evidence Retrieval (benchmark)
FinRAG	Financial Retrieval-Augmented Generation
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LLM	Large Language Model
ML	Machine Learning
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing
NoSQL	Not Only SQL
OTP	One-Time Password
PDF	Portable Document Format
PR-AUC	Precision-Recall Area Under Curve
QA	Question Answering

RAG	Retrieval-Augmented Generation
RF	Random Forest
RRF	Reciprocal Rank Fusion
ROC-AUC	Receiver Operating Characteristic -- Area Under Curve
REST	Representational State Transfer
TF-IDF	Term Frequency -- Inverse Document Frequency

CHAPTER 1

INTRODUCTION

1.1 Introduction

The Banking, Financial Services, and Insurance (BFSI) sector generates enormous volumes of heterogeneous data every day. Structured transaction records -- millions of credit and debit events flowing through payment networks -- coexist with large collections of unstructured financial documents such as annual reports, earnings releases, regulatory filings, and corporate disclosures. Together, these two data streams contain the information needed to understand financial risk, evaluate institutional health, and make informed investment decisions. However, harnessing both streams simultaneously and interactively has remained a significant technical challenge.

Traditional financial analytics platforms address either the transactional dimension or the document dimension, rarely both. Fraud-detection systems operate on transaction logs using rule engines or supervised machine learning, while document-analysis systems rely on keyword search or manual review. This separation means that an analyst investigating a suspicious transaction cannot immediately query the corresponding financial document for corroborating or contextual evidence without switching tools and reloading context.

FinRAG AI addresses this gap by providing a unified web-based intelligence platform that integrates: (1) a transaction intelligence engine powered by machine learning and anomaly detection, and (2) a financial document question-answering copilot built on Retrieval-Augmented Generation. Both engines share a common user dashboard, authentication layer, and database, and are accessible from any modern web browser without local software installation.

1.2 Background

Financial fraud detection has been studied extensively using supervised and unsupervised machine learning. Tree-based ensemble models such as Random Forest have consistently shown strong performance on tabular transaction data due to their ability to capture non-linear interactions between features while remaining interpretable through feature importances. Unsupervised anomaly detectors such as

Isolation Forest complement supervised classifiers by flagging unusual behaviour patterns even in the absence of labelled fraud examples, providing a safety net against novel attack vectors.

Retrieval-Augmented Generation (RAG) is a paradigm introduced by Lewis et al. (2020) that augments a generative language model with a non-parametric retrieval component. At inference time, relevant passages are retrieved from a document corpus and appended to the model input, grounding its response in retrieved evidence. RAG has been shown to substantially reduce hallucinations and improve factual accuracy on knowledge-intensive tasks. In the financial domain, where precise figures and regulatory language are critical, RAG is particularly well-suited because it allows the model to cite the exact passage from which a number or claim originates.

The Google Gemini API provides both generative text capabilities (via gemini-flash-lite-latest) and high-quality text embeddings (via gemini-embedding-001, 768 dimensions) used for dense retrieval. TF-IDF and BM25 provide complementary sparse retrieval that is effective for exact-match queries involving company names, financial terms, and specific numerical values.

1.3 Problem Statement

Financial institutions and individual investors face three interconnected problems:

First, transaction risk is difficult to assess in real time. Manual review of transaction logs is slow and inconsistent. While automated fraud-detection systems exist, they typically operate as black boxes without providing human-readable explanations of why a transaction was flagged, reducing analyst trust and operational efficiency.

Second, financial documents are voluminous, complex, and difficult to query. A single annual report may run to several hundred pages with dense tabular data, footnotes, and regulatory boilerplate. Analysts who need to answer specific questions -- such as the revenue growth rate for a particular segment, or the level of off-balance-sheet commitments -- must manually locate and read the relevant sections, which is time-consuming and error-prone.

Third, no readily accessible unified platform exists that allows an analyst to simultaneously investigate a flagged transaction and query the relevant financial document for supporting or contradicting evidence. The absence of such integration

forces analysts to manage multiple disconnected tools, increasing the risk of errors and missed connections.

1.4 Motivation

The motivation for FinRAG AI arises from the practical gap between what financial analysts need and what existing tools provide. Modern machine learning research has independently advanced both fraud detection and document question answering to high levels of performance, yet commercially accessible unified platforms remain rare and expensive. An open-architecture platform built on publicly available APIs and open-source libraries can demonstrate that integrated financial intelligence is technically feasible at low cost, and provides a reusable reference architecture for future development.

Additionally, the emergence of large language models with reliable API access and high-quality embedding services (such as the Google Gemini family of models) has substantially reduced the barrier to implementing production-quality RAG systems. The availability of annotated financial retrieval benchmarks such as FinDER makes it possible to evaluate retrieval quality objectively, providing a rigorous basis for comparing retrieval strategies.

1.5 Objectives

The specific objectives of the FinRAG AI project are:

1. To design and implement a hybrid transaction risk-scoring pipeline that combines a supervised RandomForestClassifier with an unsupervised IsolationForest anomaly detector, producing a calibrated risk score and human-readable risk reasons for each transaction.

2. To build a document ingestion pipeline that accepts PDF, HTML, Markdown, and plain-text financial documents, extracts structure-preserving text blocks (headings, body paragraphs, tables), and generates both heuristic and LLM-enriched metadata for each document and chunk.

3. To implement dual-mode retrieval combining BM25/TF-IDF sparse retrieval with Gemini dense-vector retrieval, fused via Reciprocal Rank Fusion, and evaluate retrieval quality using the FinDER benchmark.

4. To integrate the Google Gemini generative API for grounded answer generation, ensuring that every answer includes citations to the retrieved passages that support it.

5. To develop a multi-module web application (Flask 3.x + MongoDB) with secure authentication (bcrypt + PyJWT + OTP), providing a unified dashboard that presents both transaction analytics and document Q&A in a single interface.

6. To evaluate the system end-to-end: transaction model performance using ROC-AUC, PR-AUC, F1-score, and false-positive rate; retrieval quality using Recall@k and MRR on the FinDER benchmark.

7. To produce a deployable, maintainable application with rate limiting, request caching, compression, and structured logging, suitable for demonstration in an academic setting.

1.6 Scope of the Project

FinRAG AI is scoped as a proof-of-concept academic platform demonstrating the feasibility of unified transaction intelligence and financial document Q&A. It is designed for a single authenticated user per account, operating on personally imported transaction data and user-uploaded financial documents. It is not designed for multi-tenant institutional deployment, real-time payment interception, or regulatory compliance certification.

The transaction intelligence module operates on historical transaction data imported via CSV, Excel, PDF, or image upload. It trains a predictive model on the user's own labelled data and flags high-risk transactions for manual review. It does not connect to live payment networks or banking systems.

The RAG module processes documents uploaded by the user. It supports PDF (via pypdf), HTML, Markdown, and plain text. It does not crawl the web autonomously, though an optional SEC filing ingestion feature allows retrieval of public filings. The quality of answers depends on the quality and relevance of the uploaded documents.

1.7 Proposed Solution

FinRAG AI proposes a three-tier web application with two intelligence pipelines sharing a common backend:

The Presentation Tier consists of server-rendered HTML templates (Flask Jinja2) with Chart.js dashboards and Font Awesome icons, served over HTTP. All pages use a

consistent design language with a dark-themed sidebar navigation, glassmorphic card components, and toast notification system.

The Application Tier is a Flask 3.x application handling authentication, business logic, ML inference, and RAG query processing. It exposes a REST API consumed by the frontend JavaScript layer and integrates with the Google Gemini API for both embedding and generative functions.

The Data Tier uses MongoDB for persistent storage of users, transactions, documents, chunks, embeddings, investments, and model artifacts. The ML models (RandomForestClassifier, IsolationForest) are serialized to disk via joblib and loaded at startup.

1.8 Organization of the Report

Chapter 2 surveys the literature on financial fraud detection, anomaly detection, and retrieval-augmented generation. Chapter 3 presents the system requirements analysis. Chapter 4 describes the system architecture and design. Chapter 5 covers dataset description and data processing. Chapter 6 details the technologies and methodology. Chapter 7 describes the implementation of each module. Chapter 8 provides a detailed technical description of the RAG pipeline and transaction intelligence algorithms. Chapter 9 presents testing and performance evaluation. Chapter 10 shows system results and screenshots. Chapter 11 concludes the report and discusses future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

This chapter surveys relevant prior work across the three principal technical domains of FinRAG AI: machine learning for transaction fraud detection, unsupervised anomaly detection in financial data, and retrieval-augmented generation for document question answering. The survey also examines the limitations of existing systems and identifies the research gap that FinRAG AI addresses.

2.2 Existing Approaches to Financial Intelligence

Financial intelligence platforms have historically been designed around rule-based systems that encode expert knowledge as fixed thresholds and decision trees. While interpretable and auditable, rule-based systems suffer from brittleness: they cannot adapt to novel fraud patterns without manual updates, and they produce high false-positive rates when applied to diverse transaction populations. Statistical approaches such as logistic regression and Bayesian classifiers improved adaptability but remained limited by the assumption of feature independence and linear decision boundaries.

The introduction of ensemble learning methods -- particularly Random Forest and Gradient Boosted Trees -- significantly improved classification accuracy on imbalanced transaction datasets. These models are capable of capturing complex, non-linear feature interactions and can be trained efficiently on large datasets. Scikit-learn (Pedregosa et al., 2011) provides production-quality implementations of these algorithms, making them accessible for academic and commercial applications alike.

2.3 Machine Learning for Transaction Risk Analysis

Supervised machine learning for fraud detection frames the problem as a binary classification task: given a set of transaction features, predict whether the transaction is fraudulent (1) or legitimate (0). The key challenges are (a) severe class imbalance -- fraud events typically represent less than 1% of all transactions -- and (b) the need for real-time or near-real-time inference. Random Forest addresses class imbalance through the class_weight="balanced_subsample" parameter, which adjusts sample weights within each tree's bootstrap sample proportionally to class frequencies. This

avoids the majority-class dominance that would otherwise bias predictions toward "not fraud".

Threshold selection is also critical in imbalanced classification. A fixed 0.5 decision threshold often yields poor recall on the minority class. In FinRAG AI, the decision threshold is selected by maximising the fraud-class F1-score on out-of-fold predictions of the training split, following the methodology recommended in the scikit-learn documentation. This approach avoids test-set leakage while producing a threshold that is calibrated to the actual prevalence of fraud in the training data.

2.4 Anomaly Detection in Financial Transactions

Isolation Forest (Liu et al., 2008) is an unsupervised anomaly detection algorithm based on the observation that anomalous observations are few and different, and are therefore easier to isolate than normal ones. The algorithm builds an ensemble of random isolation trees by recursively partitioning the feature space; the isolation depth of a point (the number of splits required to isolate it) is inversely related to its anomaly score. Points that are isolated in fewer splits receive higher anomaly scores.

In FinRAG AI, Isolation Forest is trained exclusively on legitimate (non-fraud) transactions to learn the distribution of normal behaviour. At inference time, any transaction -- whether labelled or not -- that deviates significantly from learned normal patterns receives a high anomaly score. This is important because it enables the system to detect fraud patterns that were not present in the training labels, supplementing the supervised classifier's coverage. The two scores are combined as: risk_score = 0.75 x fraud_probability + 0.25 x anomaly_score.

2.5 Retrieval-Augmented Generation

Lewis et al. (2020) introduced Retrieval-Augmented Generation (RAG), a paradigm that combines a retrieval component (a non-parametric dense passage retriever) with a generative language model. At inference time, the retriever selects the k most relevant passages from a large document corpus, which are then prepended to the generative model's input. This grounding substantially reduces hallucinations on knowledge-intensive tasks and enables attribution of claims to source passages.

Subsequent work has extended RAG in several directions: hybrid retrieval combining dense and sparse methods (Formal et al., 2021), metadata-enriched chunks to improve precision, reranking with cross-encoders, and multi-hop reasoning over multiple

documents. FinRAG AI implements hybrid retrieval by fusing TF-IDF/BM25 sparse scores with Gemini dense-vector scores using Reciprocal Rank Fusion (RRF). Metadata enrichment is performed at both document and chunk level, providing thematic cluster labels, entity lists, and answered-question templates that are prepended to each chunk text before embedding, improving semantic alignment with natural-language queries.

2.6 Financial Document Question Answering

Financial QA is a challenging sub-task of open-domain QA because financial documents combine structured tabular data (income statements, balance sheets) with densely worded narrative text and regulatory boilerplate. Early financial QA systems focused on structured data extraction and were unable to answer questions that required reasoning over narrative text. The introduction of pre-trained language models (BERT, RoBERTa) improved performance on unstructured financial QA, but these models are limited to passages short enough to fit within their context windows.

RAG-based approaches for financial documents overcome context-length limitations by retrieving only the most relevant chunks. The FinDER benchmark (Financial Document Evidence Retrieval) provides 630 expert-annotated evidence passages from 10-K filings and evaluates retrieval systems on whether the exact annotated passage is retrieved at various rank cutoffs. FinRAG AI uses FinDER to evaluate its retrieval component; the dense retrieval mode (gemini-embedding-001) achieves Recall@10 of 88% and MRR of 0.778 on a 25-query random sample (seed 7).

2.7 Existing System and Limitations

Existing financial analytics platforms such as Bloomberg Terminal and Refinitiv Eikon provide comprehensive market data and document access, but require institutional subscriptions costing tens of thousands of dollars per year. Open-source fraud detection tools exist (e.g., PyOD for anomaly detection) but do not provide web-based interfaces or document Q&A capabilities. General-purpose RAG frameworks such as LangChain and LlamaIndex support financial document processing but require significant integration work and do not include transaction analytics modules. No single accessible platform integrates transaction risk scoring, anomaly detection, financial document question answering, portfolio management, and EMI calculation in a unified interface.

2.8 Research Gap

The survey reveals a clear gap: integrated financial intelligence platforms that combine (a) hybrid ML+anomaly transaction risk scoring with (b) metadata-driven RAG for financial document Q&A within (c) a unified, accessible web interface do not exist as open, academic systems. FinRAG AI fills this gap by delivering a complete implementation of both components, evaluated against established benchmarks, and packaged as a deployable web application.

2.9 Summary

This chapter surveyed supervised fraud detection (Random Forest, Isolation Forest), retrieval-augmented generation (Lewis et al. 2020), hybrid retrieval (TF-IDF + BM25 + dense vectors), and financial QA benchmarks (FinDER). The identified research gap -- the absence of a unified, accessible platform integrating transaction intelligence and financial RAG -- directly motivates the design of FinRAG AI, described in subsequent chapters.

CHAPTER 3

SYSTEM STUDY AND REQUIREMENTS ANALYSIS

3.1 Problem Definition

The core problem is the lack of a unified, accessible platform for financial intelligence that can simultaneously (a) assess the risk of individual transactions using trained ML models and (b) answer natural-language questions about financial documents using retrieved evidence. Analysts must currently use multiple disconnected tools, each requiring separate authentication, data ingestion, and workflow management.

3.2 Existing System

Current financial intelligence tools fall into two categories: (1) enterprise platforms (Bloomberg, Refinitiv) that are comprehensive but expensive and inaccessible to students and small organisations; and (2) single-purpose open-source tools that address either fraud detection or document analysis but not both. General-purpose RAG frameworks require extensive customisation to handle financial documents and provide no transaction analytics capability. No integrated open-architecture solution exists.

3.3 Limitations of Existing System

1. Cost: Enterprise financial intelligence platforms are prohibitively expensive for academic and small-scale use.
2. Fragmentation: Separate tools for fraud detection, document search, and portfolio management require analysts to manually cross-reference results across platforms.
3. Opaque risk scores: Many fraud detection systems provide binary flags without explanation, reducing analyst trust.
4. No document grounding: Fraud detection tools do not integrate with financial documents, preventing evidence-based investigation.
5. Hallucination in LLM-only systems: LLMs without retrieval are prone to fabricating financial figures, which is unacceptable in a regulated environment.

3.4 Proposed System

FinRAG AI proposes an integrated web application with two intelligence pipelines -- a hybrid ML+anomaly transaction risk engine and a metadata-driven RAG document

copilot -- operating through a unified Flask backend with MongoDB persistence, Chart.js dashboards, and a secure JWT+OTP authentication layer.

3.5 Advantages of Proposed System

1. Integration: Both transaction analytics and document Q&A are available in a single interface.
2. Explainability: Risk scores are accompanied by human-readable risk reasons.
3. Evidence grounding: Every RAG answer cites the source passage and document, making it auditable.
4. Accessibility: The system runs on commodity hardware and uses free-tier API access during development.
5. Extensibility: The modular Flask architecture allows new analytics modules to be added independently.

3.6 Functional Requirements

ID	Requirement	Priority
FR01	The system shall allow users to register with email, name, and 6-digit PIN and verify via OTP.	High
FR02	The system shall authenticate users via email and	High

	6-digit PIN with JWT token issuance.	
FR03	The system shall allow import of transaction data from CSV, Excel, PDF, and image files.	High
FR04	The system shall train a hybrid fraud detection model (RandomForest + IsolationForest) on imported data.	High
FR05	The system shall assign a risk score (0-100) and severity level to each transaction.	High
FR06	The system shall display fraud alerts with severity classification (Critical/High/Medium/Low).	High
FR07	The system shall allow users to upload financial documents (PDF, HTML, MD, TXT) for RAG indexing.	High
FR08	The system shall answer natural-language questions about uploaded documents with cited passages.	High
FR09	The system shall provide an interactive dashboard with transaction volume, spending, and forecast charts.	Medium
FR10	The system shall allow users to manage an investment portfolio with AI allocation analysis.	Medium
FR11	The system shall calculate EMI and assess loan affordability based on user-provided income.	Medium
FR12	The system shall provide a credit health score based on transaction behaviour and KYC status.	Medium

3.7 Non-Functional Requirements

ID	Requirement	Metric
NFR01	Performance: Single-transaction risk scoring	< 100 ms

	shall complete in under 100 ms.	
NFR02	Availability: The application shall be available during active development sessions.	Development-grade
NFR03	Security: All API endpoints shall require a valid JWT token; passwords stored as bcrypt hashes.	JWT + bcrypt
NFR04	Scalability: The Flask application shall support at least 10 concurrent users without error.	10 concurrent users
NFR05	Maintainability: The codebase shall be organised in a single-file Flask application with clear section comments.	Section-documented

3.8 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th generation) or equivalent
RAM	8 GB (16 GB recommended for model training)
Storage	5 GB free space for application, models, and database
GPU	Not required (all ML inference runs on CPU)
Network	Internet connection required for Gemini API calls

3.9 Software Requirements

Category	Technology	Version

Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS 12+	Any
Programming Language	Python	3.10+
Web Framework	Flask	>=3.0
Database	MongoDB Atlas (cloud) or local MongoDB	6.x+
ML Library	scikit-learn	>=1.5
Data Processing	pandas, numpy	2.2.3 / <2.0
LLM / Embedding API	Google Gemini (google-genai)	>=1.0
PDF Parsing	pypdf	>=5.0
Authentication	bcrypt, PyJWT	>=4.0, >=2.8
Model Persistence	joblib	Latest
WSGI Server	gunicorn	>=21.2
Browser	Google Chrome 100+ or Firefox 100+	Any modern

3.10 Feasibility Analysis

Technical Feasibility: All required technologies are available as open-source libraries or accessible APIs. The Google Gemini API provides free-tier access suitable for academic projects. MongoDB Atlas offers a free shared cluster sufficient for the data volumes expected in this project. The Flask web framework is well-documented and widely used for academic projects of this scope. Technical feasibility is confirmed.

Economic Feasibility: The project uses exclusively free or open-source tools. The primary operational cost is the Google Gemini API, which provides free-tier access for low-volume usage. No commercial software licenses are required. Economic feasibility is confirmed.

Operational Feasibility: The system is operated through a standard web browser with no client-side installation required. The Flask application can be launched locally with a single command. Operational feasibility is confirmed.

3.11 Use Cases and User Workflows

Primary Use Case 1 -- Import and analyse transactions: A user uploads a CSV of bank transactions, the system parses and maps columns, trains the ML model, and presents a dashboard with risk-scored transactions and fraud alerts.

Primary Use Case 2 -- Financial document Q&A: A user uploads an annual report PDF, the system extracts and indexes its content, and the user asks natural-language questions such as "What was the net profit margin in FY2024?" and receives cited answers.

Secondary Use Case -- Portfolio management: A user adds investment holdings (stocks, mutual funds, bonds), the system displays allocation charts and triggers an AI portfolio analysis via the Gemini API.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

FinRAG AI follows a three-tier architecture: a Presentation Tier (browser), an Application Tier (Flask), and a Data Tier (MongoDB + disk-based ML models). The browser communicates with the Flask backend via HTTP REST calls, receiving server-rendered HTML pages and JSON API responses. The Flask backend orchestrates business logic, ML inference, and external API calls. MongoDB stores all persistent state. Figure 4.1 illustrates the overall architecture.

[FIGURE 4.1: Overall System Architecture -- Three-tier diagram showing Browser, Flask Application (Auth, ML Engine, RAG Engine, REST API), MongoDB, and Gemini API]

4.2 High-Level Architecture

At the highest level, FinRAG AI comprises four subsystems: (1) Authentication and User Management; (2) Transaction Intelligence; (3) Financial RAG; and (4) Supporting Financial Modules (investments, loans, credit health, budget). Each subsystem has its own set of Flask routes, MongoDB collections, and business logic, but all share a common JWT authentication middleware, logging infrastructure, and response formatting utilities.

4.3 Frontend Architecture

The frontend is rendered server-side via Flask Jinja2 templates. Each page template extends a common layout injected by navbar.js -- a JavaScript file loaded on every page that provides the sidebar navigation, global toast notification system (showToast), styled confirmation modals (showConfirm), and JWT session expiry detection. Chart.js (loaded via CDN) renders all analytics charts. Font Awesome provides icons throughout the interface.

Client-side JavaScript handles API calls (via fetch with JWT Bearer tokens in the Authorization header), AbortController-based upload cancellation, CSV export, and dynamic chart rendering. All pages follow a dark-themed glassmorphic design with responsive breakpoints at 768 px and 480 px.

4.4 Backend Architecture

The application backend is a single Flask 3.x application (app.py) structured with clear section comments delineating functional areas. Flask extensions used include: Flask-CORS for cross-origin support, Flask-Limiter for API rate limiting, Flask-Caching for response caching, and Flask-Compress for gzip compression. A custom auth_required decorator validates the JWT token from the Authorization header and attaches the decoded user payload to request.user for downstream handlers.

Email sending (for OTP delivery) uses smtplib with TLS, with SMTP credentials loaded from environment variables via python-dotenv. All endpoints return JSON responses with consistent status codes and error message formats.

4.5 Transaction Intelligence Architecture

The transaction intelligence pipeline has three phases: data ingestion, model training, and inference. During ingestion, raw transaction files (CSV, Excel, PDF, image) are parsed and normalised to a canonical schema. Column mapping handles varying header names across different bank export formats. During training, the normalised dataset is passed to train_models(), which produces a RandomForestClassifier, a LabelEncoder for the channel feature, and an anomaly bundle containing the IsolationForest and calibration metadata. Models are serialised to disk via joblib. During inference, score_transactions() applies the trained models to produce per-transaction risk scores, classifications, anomaly scores, and risk reason codes.

4.6 Financial RAG Architecture

The RAG pipeline operates in two phases: indexing and querying. During indexing, an uploaded document is parsed into a list of structure-preserving blocks (headings, body text, tables). Blocks are assembled into overlapping chunks, each annotated with section context. Heuristic and Gemini-generated metadata (company name, document type, fiscal year, thematic clusters) are attached to the document record. Each chunk is enriched with cluster labels, entity lists, and answered-question templates via a batch Gemini JSON call. Chunks are then embedded using the Gemini Embedding API (gemini-embedding-001) and stored in MongoDB alongside their embedding vectors and metadata.

During querying, the user question is matched against indexed chunks using both sparse (TF-IDF + BM25) and dense (cosine similarity of Gemini embeddings) retrievers. Results are fused via Reciprocal Rank Fusion. The top-k chunks are

assembled into a prompt that includes the question and retrieved context, and the Gemini generative model produces a grounded answer with passage citations.

[FIGURE 4.3: RAG Pipeline Architecture -- Upload, Parse, Chunk, Enrich, Embed, Store; Query, Sparse+Dense Retrieval, RRF Fusion, LLM Answer Generation]

4.7 Database Design

MongoDB is used as the primary data store. The key collections and their principal fields are described in the table below.

Collection	Key Fields	Purpose
users	_id, name, email, password_hash, is_verified, kyc_status, created_at	User accounts and authentication
transactions	_id, user_id, transaction_id, amount, channel, timestamp, category, is_fraud, risk_score, risk_level, import_batch_id	Transaction records and risk annotations
rag_documents	_id, user_id, filename, file_type, metadata, chunk_count, indexed_at	Uploaded document records and metadata
rag_chunks	_id, document_id, user_id, text, section, embedding, metadata	Text chunks with embeddings for retrieval
investments	_id, user_id, asset_name, asset_type, quantity, purchase_price, current_price	Investment portfolio holdings
rag_conversations	_id, user_id, question, answer, sources, created_at	RAG Q&A conversation history

[FIGURE 4.4: MongoDB Database Schema -- Entity relationship showing collections and key fields]

4.8 Data Flow Diagram

Figure 4.2 shows the high-level data flow. User input (transaction file or document) enters the system at the upload endpoint. Transaction files flow through parsing, column mapping, normalisation, model training/scoring, and storage in the transactions collection. Document files flow through text extraction, chunking, enrichment, embedding, and storage in the rag_documents and rag_chunks collections. Query input flows through the RAG retrieval pipeline and returns a grounded answer with citations.

[FIGURE 4.2: Data Flow Diagram -- Upload, Parse, Process (ML/RAG), Store, Dashboard / Answer]

4.9 API Architecture

The REST API is organised into functional groups. Authentication endpoints (POST /register, POST /login, POST /verify-registration, POST /forgot-password, POST /reset-password) handle the full authentication lifecycle. Transaction endpoints (POST /upload, POST /upload/confirm, GET /api/transactions, POST /api/transactions/{id}/dispute) manage transaction data. RAG endpoints (POST /api/rag/index, POST /api/rag/ask, GET /api/rag/documents) handle document indexing and question answering. Dashboard and analytics endpoints (GET /api/dashboard, GET /api/metrics, GET /api/alerts) serve aggregated data to the frontend.

4.10 Security and Error Handling

Authentication uses bcrypt-hashed passwords (cost factor 12) and 6-digit numeric PINs. JWT tokens carry a configurable expiry (default 24 hours); the client monitors the token exp claim and displays a session-expiry warning 5 minutes before expiration. OTP delivery for email verification and account deletion uses time-limited codes via SMTP. Flask-Limiter enforces per-IP rate limits on authentication and inference endpoints to mitigate brute-force and denial-of-service attacks. Prompt injection detection regex guards all user inputs passed to the Gemini API.

4.11 System Workflow

A typical user session proceeds as: (1) register with email, name, and 6-digit PIN; (2) verify email via OTP; (3) log in, receiving a JWT token stored in localStorage; (4) import transactions via CSV upload, triggering column mapping, confirmation, and model training; (5) review the dashboard for risk scores and fraud alerts; (6) upload a

financial document for RAG indexing; (7) ask questions in the FinRAG Copilot and receive cited answers; (8) explore supporting modules (investments, loans, credit health) as needed.

CHAPTER 5

DATASET AND DATA PROCESSING

5.1 Introduction

FinRAG AI operates on two distinct types of data: structured transaction records and unstructured financial documents. This chapter describes the format, preprocessing pipeline, and privacy considerations for each data type, along with the feature engineering performed before ML model training and the text processing performed before RAG indexing.

5.2 Transaction Data

Transaction data is imported by the user from their own bank export files. The system accepts CSV, Excel (.xlsx/.xls), PDF bank statements, and image files (JPG, PNG, BMP, TIFF). The canonical schema includes the following columns:

Column	Type	Description
transaction_id	String	Unique identifier for the transaction
customer_id	String	Identifier of the account holder
transaction_amount	Float	Transaction value in Indian Rupees (INR)
transaction_date	String (ISO 8601)	Date and time of the transaction

channel	Categorical	Payment channel (UPI, ATM, NEFT, IMPS, etc.)
category	String	Spending category (Groceries, Dining, etc.)
description	String	Transaction narrative
is_fraud	Binary (0/1)	Ground-truth fraud label (where available)
account_age_days	Integer	Age of the account at transaction time
kyc_verified	Binary (0/1)	Whether the account has passed KYC verification

The system does not require all columns to be present; the column-mapping interface allows the user to match their export headers to the canonical schema. Missing values in numeric columns are filled with the column median; missing categorical values are filled with "Unknown".

5.3 Financial Document Sources and Formats

Users upload financial documents they wish to query via the FinRAG Copilot. Supported formats are: PDF (.pdf), HTML/HTM (.html, .htm), Markdown (.md), and plain text (.txt). In addition, the system can ingest SEC filings from the EDGAR full-text search API via the /api/rag/ingest/sec endpoint and public documents via URL via /api/rag/ingest/url.

5.4 Data Preprocessing

For transaction data, preprocessing involves: (a) normalising column names to the canonical schema via the /upload/map endpoint; (b) converting transaction_amount to float and clamping negative values; (c) parsing transaction_date to a timezone-aware datetime object; (d) encoding the channel field as an integer via sklearn LabelEncoder (fitted per training run); and (e) computing account_age_days from account_creation_date if not provided.

For document data, preprocessing involves: (a) extracting raw text from the source format; (b) segmenting into structure-preserving blocks; (c) assembling blocks into overlapping chunks; and (d) generating metadata. Each stage is described in detail in Sections 5.7 through 5.9.

5.5 Data Validation and Cleaning

Transaction data validation checks that: transaction_amount is positive; transaction_date is parseable and not in the future; channel values are drawn from the known set or classified as "Unknown". Rows that fail date parsing or have non-numeric amounts are placed in an invalid-CSV report rather than silently dropped, allowing the user to correct their source data. Transactions with is_fraud values other than 0 or 1 are used for risk scoring but excluded from model training.

5.6 Feature Engineering

The four features used by the ML model are: transaction_amount (raw, log1p-transformed for anomaly scoring), channel (label-encoded integer), account_age_days (raw, log1p-transformed for anomaly scoring), and kyc_verified (binary 0/1). Log transformations are applied only within the anomaly scorer's _anomaly_features() function to normalise skewed distributions. The Random Forest receives raw (non-transformed) values. No additional engineered features are added; the small feature set is intentional to keep the model interpretable and its requirements minimal.

5.7 Document Parsing and Text Extraction

PDF documents are parsed using pypdf's PdfReader. The parser extracts text page by page and applies a structure-detection heuristic that classifies short, bold-style lines as headings and longer lines as body text. Tables within PDFs are detected by the presence of | separators or whitespace-aligned columns and are preserved as single text blocks (rows joined with " | ") to maintain the association between values and their row/column labels. HTML documents are parsed with Python's built-in html.parser, stripping script/style tags and preserving heading hierarchy. Markdown documents are parsed line-by-line, preserving ATX headings and table rows.

5.8 Text Chunking and Metadata Processing

Extracted text blocks are assembled into overlapping chunks. The chunker groups consecutive blocks until a character limit is reached, then starts a new chunk with overlap from the previous chunk to preserve context across boundaries. Each chunk retains the section heading under which it appears. Metadata is generated at two levels: document-level metadata (company name, document type, fiscal year, thematic cluster labels, currency) and chunk-level metadata (parent clusters, named entities, answered_questions and retrieval_nuggets). When a Gemini API key is available, an

LLM-based metadata enrichment call refines the document summary and cluster labels.

5.9 Embedding and Indexing

Each chunk's text is concatenated with its metadata fields (cluster labels, entity list, answered questions) before embedding to improve semantic alignment with natural-language queries. Embeddings are generated in batches of up to 40 texts per API call using the Gemini Embedding API (gemini-embedding-001, task_type=RETRIEVAL_DOCUMENT, output_dimensionality=768). Embeddings are stored as lists of floats in the rag_chunks MongoDB collection alongside the chunk text and metadata. At query time, the question is embedded with task_type=RETRIEVAL_QUERY, and cosine similarity is computed in memory by iterating over stored chunk vectors.

5.10 Data Privacy

All transaction data and uploaded documents are stored within the user's own MongoDB database and are accessible only to that user (enforced by per-user query filtering on user_id). No transaction data or document content is sent to external services other than the Gemini API for embedding and answer generation. Users are advised to avoid uploading documents containing personally identifiable information beyond what is necessary for their analysis.

CHAPTER 6

TECHNOLOGIES AND METHODOLOGY

6.1 Programming Languages

Language	Usage in FinRAG AI
Python 3.10+	Backend application logic, ML pipeline, RAG pipeline, API handlers
JavaScript (ES2020)	Frontend interactivity, Chart.js rendering, API calls, DOM manipulation
HTML5	Page structure and semantic markup via Jinja2 templates
CSS3	Styling, responsive layout, glassmorphic design, animations

6.2 Frontend Technologies

Technology	Version	Purpose
Flask Jinja2	3.x	Server-side HTML template rendering
Chart.js	CDN latest	Dashboard charts: bar, line, doughnut, gauge
Font Awesome	6.4.0	Icon library used throughout the interface
Custom CSS	N/A	Glassmorphic dark theme, responsive sidebar, toast system

Fetch API	Browser native	Asynchronous REST API calls with JWT bearer tokens

6.3 Backend Technologies

Library	Version	Purpose
Flask	>=3.0	Web framework, routing, request/response handling
Flask-CORS	>=6.0	Cross-Origin Resource Sharing
Flask-Limiter	>=3.5	Per-IP rate limiting for auth and inference endpoints
Flask-Caching	>=2.3	Response caching to reduce database and API load
Flask-Compress	>=1.15	Gzip response compression
python-dotenv	>=1.0	Environment variable management (.env file loading)

PyJWT	>=2.8	JWT token encoding and decoding
bcrypt	>=4.0	Password hashing and verification
PyMongo	>=4.6	MongoDB driver for Python
pypdf	>=5.0	PDF text extraction
Pillow	>=10.0	Image processing for image-based transaction import
openpyxl	Latest	Excel (.xlsx) file parsing
python-dateutil	>=2.8	Flexible date parsing
gunicorn	>=21.2	WSGI production server
smtplib	stdlib	Email delivery for OTP codes

6.4 Machine Learning Frameworks and Libraries

Library	Version	Purpose
scikit-learn	>=1.5	RandomForestClassifier, IsolationForest, LabelEncoder, StratifiedKFold, metrics
pandas	2.2.3	Data loading, cleaning, and feature preparation
numpy	<2.0	Numerical computation, embedding similarity, array operations
joblib	Latest	Model serialisation (pickle-compatible, optimised for numpy arrays)
sklearn.feature_extraction	Part of scikit-learn	TfidfVectorizer for sparse retrieval

6.5 Transaction Risk Detection Algorithms

The transaction risk pipeline uses two complementary algorithms:

RandomForestClassifier: An ensemble of 200 decision trees trained on the four transaction features (amount, channel, account age, KYC status). Class balancing is applied via class_weight="balanced_subsample". The decision threshold is selected by maximising the fraud-class F1-score on out-of-fold cross-validation predictions, avoiding any optimisation on the test split. The model outputs a fraud probability in [0, 1] for each transaction.

IsolationForest: An ensemble of 200 isolation trees fitted exclusively on legitimate (non-fraud) training transactions. For each new transaction, the algorithm computes an anomaly score based on the mean isolation depth across trees. Scores are normalised to [0, 1] using the 1st and 99th percentile bounds observed on the training data. Transactions that deviate from learned normal patterns receive scores close to 1.

The final risk score is: risk_score = clip(100 x (0.75 x fraud_prob + 0.25 x anomaly_score), 0, 100). A transaction with risk_score > 70 is labelled Critical, 50-70 High, 30-50 Medium, and below 30 Low.

6.6 Retrieval-Augmented Generation

The RAG pipeline follows the architecture of Lewis et al. (2020) extended with metadata enrichment and hybrid retrieval. The pipeline has five stages: (1) document ingestion and chunking; (2) metadata generation (heuristic + LLM); (3) embedding generation (Gemini); (4) hybrid retrieval (TF-IDF+BM25 sparse + dense cosine similarity, fused via RRF); and (5) answer generation (Gemini generative model with retrieved context in the prompt).

6.7 Embedding Models and Retrieval Techniques

Sparse retrieval uses a TfidfVectorizer (sklearn, bigrams, sublinear TF, max 50000 features) combined with a custom BM25 scorer built on per-document term frequency statistics. Both sparse scores are fused via Reciprocal Rank Fusion before being combined with the dense scores. Dense retrieval uses Gemini Embedding Model gemini-embedding-001 (768-dimensional output embeddings). Cosine similarity is computed between the query embedding and each stored chunk embedding using in-memory numpy dot products.

Retrieval results from sparse and dense modes are combined using Reciprocal Rank Fusion (RRF), which assigns each candidate a score of 1/(rank + k) summed across all

rank lists. RRF is known to be robust to varying score scales and to improve recall relative to either individual retriever.

6.8 Large Language Model Integration

FinRAG AI uses the Google Gemini API (google-genai Python SDK >=1.0) for two purposes: embedding generation (model: gemini-embedding-001) and text generation (model: gemini-flash-lite-latest, configurable via GEMINI_MODEL environment variable). Text generation is used for: (a) document and chunk metadata enrichment (JSON output mode); (b) financial question answering (streaming and non-streaming); (c) AI portfolio analysis; (d) fraud transaction analysis. A shared retry wrapper (_with_retry) handles rate-limit (HTTP 429 / RESOURCE_EXHAUSTED) and transient server errors with exponential backoff up to 6 retries.

6.9 Database and Storage Technologies

MongoDB (via PyMongo >=4.6) is used as the primary database. MongoDB's flexible document schema is well-suited for storing heterogeneous chunk metadata, variable-length embedding vectors, and nested transaction annotation objects. The application connects to MongoDB Atlas (cloud) by default; a local MongoDB instance can be used by setting the MONGO_URI environment variable. Model artifacts (fraud_model.pkl, encoder.pkl, anomaly_model.pkl) are stored on disk in the assets/models/ directory and loaded at application startup.

6.10 Development Tools

Tool	Purpose
Visual Studio Code	Primary development environment

Git / GitHub	Version control and code hosting
Postman	API endpoint testing
MongoDB Compass	Database inspection and query testing
pytest	Automated test suite (tests/ directory)
python-dotenv	Environment variable management
pyarrow	Parquet file support for FinDER evaluation dataset

6.11 Rationale for Technology Selection

Flask was chosen over Django for its lightweight, un-opinionated architecture, which is well-suited to a project that requires fine-grained control over request handling and streaming responses. MongoDB was chosen over a relational database because the schema of transaction records, document chunks, and embedding vectors varies per user and does not fit neatly into a fixed relational schema. The Google Gemini API was chosen for its combination of high-quality embeddings, generative capability, and accessible free tier. scikit-learn was chosen for the ML pipeline because its implementations of RandomForest and IsolationForest are production-grade, well-documented, and require no GPU infrastructure.

CHAPTER 7

IMPLEMENTATION AND MODULE DESCRIPTION

7.1 System Implementation Overview

FinRAG AI is implemented as a single-file Flask application (app.py, approximately 7,000 lines) supported by HTML templates (templates/), static assets (static/css/, static/js/, static/images/), model artifacts (assets/models/), and a test suite (tests/). The application is configured via a .env file and started with gunicorn or the Flask development server.

7.2 Project Structure

Path	Description
app.py	Flask application: all routes, ML pipeline, RAG pipeline, auth, utilities
requirements.txt	Python package dependencies
.env	Environment variables: MONGO_URI, GEMINI_API_KEY, JWT_SECRET, SMTP credentials

templates/	Jinja2 HTML templates: auth.html, dashboard.html, transactions.html, upload.html, fraudalerts.html, rag.html, investments.html, loans.html, credit_score.html, insights.html, predictive.html, profile.html
static/css/styles.css	Global CSS: dark theme, glassmorphic cards, toast, confirm modal, breadcrumbs, skeleton loaders
static/js/navbar.js	Sidebar navigation, toast/confirm system, session expiry detection
static/js/dashboard.js	Dashboard chart rendering and API data loading
assets/models/	Serialised ML models: fraud_model.pkl, encoder.pkl, anomaly_model.pkl
assets/logs/app.log	Application log file
static/samples/	Sample transaction CSV for demonstration
tests/	pytest test files: test_finrag_pipeline.py, test_transaction_intel.py, etc.
evaluation/	FinDER retrieval evaluation: results/finder_latest.md

7.3 Frontend Implementation

The frontend consists of 12 template pages, each served by a dedicated Flask route:

Dashboard (/dashboard): Displays five metric cards (total transactions, model-flagged transactions, disputed transactions, security status, net cash flow), a bento-grid analytics layout with transaction volume, trust score, income vs. expense, spending categories, fraud trend, and spending forecast charts, a recurring payments detector, and a recent alerts preview.

Transactions (/transactions): A paginated, filterable table of all imported transactions with risk score badges, dispute and delete actions, date-range filtering, and a client-side CSV export function.

Upload (/upload-page): A drag-and-drop file uploader supporting CSV, Excel, PDF, and image formats. An AbortController allows cancellation mid-upload.

Fraud Alerts (/fraud-alerts): A list of model-flagged transactions with severity-coloured cards (Critical, High, Medium, Low), an "Ask AI" button for each alert that triggers a Gemini-powered fraud analysis.

FinRAG Copilot (/rag): A three-panel interface (document library, chat, sources). Users upload documents, ask questions, and receive streamed answers with cited passage excerpts.

Predictive Analytics (/predictive): An interactive form for predicting fraud probability for a hypothetical transaction, a model metrics dashboard (ROC-AUC, PR-AUC, F1), a retrain button, and a metrics history chart.

Credit Health (/credit-score): A simulated credit health score with explanation cards and improvement recommendations.

Investments (/investments): A portfolio management interface with total value, holdings list, allocation doughnut chart, and AI portfolio analysis.

Loans & EMI (/loans): An interactive EMI calculator with loan amount, interest rate, and tenure sliders.

AI Model Insights (/insights): A table of retrieval evaluation metrics from the FinDER benchmark runs.

Profile (/profile): User profile management with editable fields, profile picture upload, password change, and account deletion (OTP-verified).

7.4 Backend Implementation

The Flask backend is organised into logical sections within app.py: GEMINI CLIENT (shared LLM/embedding wrapper), DOCUMENT PARSING (structure-preserving text extraction), METADATA (heuristic and LLM-based document/chunk metadata), RETRIEVAL (TF-IDF, BM25, dense, hybrid), CONVERSATION (RAG Q&A pipeline), TRANSACTION INTELLIGENCE (training and scoring), APP (Flask application setup, routes, authentication).

7.5 Document Upload and Processing

Document upload is handled by the POST /upload endpoint, which accepts multipart form data. The file is saved to the uploads/ directory, and a background parsing job extracts blocks, assembles chunks, generates metadata, and stores the results. Progress is communicated via the /api/rag/documents listing. When a Gemini API key is available, metadata enrichment runs as a background thread. The POST /api/rag/index endpoint triggers embedding generation for all un-indexed chunks of a document.

7.6 Financial Question-Answering Module

The FinRAG Copilot is implemented at POST /api/rag/ask (non-streaming) and POST /api/rag/ask/stream (server-sent events streaming). On receiving a query, the handler: (1) embeds the query with task_type=RETRIEVAL_QUERY; (2) runs sparse retrieval (TF-IDF + BM25) against a per-user in-memory sparse index; (3) runs dense retrieval by computing cosine similarities against stored chunk embeddings; (4) fuses results via RRF; (5) assembles the top-k chunk texts into a prompt; (6) calls the Gemini generative model. The response includes the answer text and a sources array listing the document name, chunk section, and page number for each cited passage.

7.7 Transaction Intelligence Module

Transaction import is a two-step process: (1) POST /upload parses the file and returns a column-mapping proposal; (2) POST /upload/confirm applies the confirmed column mapping, normalises the data, scores each transaction against the loaded model (if available), and stores the results in MongoDB. Model training is triggered by POST /api/retrain, which fetches all labelled transactions for the user, calls train_models(), serialises the resulting artifacts to disk, and reloads them into the global ml_model, encoder, and anomaly_bundle variables.

7.8 Risk Scoring and Anomaly Detection

The score_transactions() function applies the loaded RandomForestClassifier and IsolationForest to a DataFrame of transactions. For each transaction it computes: fraud_probability (float in [0,1]), anomaly_score (float in [0,1]), risk_score (integer 0-100), risk_level (Critical/High/Medium/Low/Normal), and risk_reasons (list of human-readable strings identifying which features contributed to the risk, e.g., "Unusually high transaction amount", "New account (age < 30 days)").

7.9 API Integration

All API endpoints return JSON with a consistent envelope: {"status": "ok" or "error", "data": ..., "error": "message"}. Authentication is enforced by the @auth_required decorator, which extracts the Bearer token from the Authorization header, decodes it with PyJWT, and attaches the decoded payload to request.user. Token expiry is handled by returning HTTP 401 with {"error": "Token expired"}, which the client detects and redirects to the login page.

7.10 Database Integration

MongoDB is accessed via a lazy-initialised get_db() function that creates and caches a MongoClient connection. All queries include a user_id filter to enforce data isolation between users. Upsert operations (update_one with upsert=True) are used for import batches to prevent duplicate records. Compound indexes on (user_id, transaction_id) and (document_id, chunk_index) ensure fast retrieval.

7.11 Validation and Error Handling

Input validation is performed at the API boundary: numeric fields are coerced via pd.to_numeric with errors="coerce"; missing required fields return HTTP 400 with descriptive error messages; file uploads check extension and MIME type before processing. The frontend supplements server-side validation with client-side checks (regex for PIN format, date-range validation) that use the showToast() notification rather than native alert(). All server-side exceptions are caught, logged at ERROR level, and returned to the client as structured JSON error responses.

[FIGURE 7.1: Main Dashboard -- Screenshot showing stat cards, bento-grid charts, and recent alerts]

[FIGURE 7.2: Transaction Table -- Screenshot showing risk badges, pagination, and export button]

[FIGURE 7.3: FinRAG Copilot -- Screenshot showing document library, question input, and cited answer]

CHAPTER 8

RAG PIPELINE AND TRANSACTION INTELLIGENCE

8.1 Overview of the Intelligence Pipeline

FinRAG AI's intelligence capabilities are delivered by two interoperating pipelines: the Financial RAG Engine, which processes documents and answers questions, and the Transaction Intelligence Engine, which scores and classifies transactions. Both pipelines share the Gemini API client, the MongoDB connection, and the Flask authentication layer.

[FIGURE 8.1: RAG Pipeline Workflow -- Sequential steps from document upload to answer generation]

8.2 Document Ingestion

The document ingestion stage accepts files in four formats: PDF (.pdf, via pypdf PdfReader), HTML/HTM (via Python html.parser), Markdown (.md, via line-by-line parser), and plain text (.txt, direct read). The ALLOWED_EXTENSIONS set in app.py defines the accepted extensions. Files outside this set are rejected at the upload endpoint with an HTTP 400 error.

8.3 Text Extraction and Chunking

Each format-specific parser produces a list of blocks, where each block is a dictionary with keys: type (heading, text, or table), text (string content), page (integer page number, None for non-PDF), and units (financial unit annotation, e.g., "millions"). The chunker iterates over the block list and assembles chunks of approximately CHUNK_SIZE characters (configurable), with an overlap of CHUNK_OVERLAP characters to preserve cross-boundary context. Each chunk inherits the section heading from the most recently encountered heading block. Tables are kept as atomic units and not split across chunks.

8.4 Metadata Generation

Metadata is generated at two levels. At the document level, heuristic_document_metadata() extracts: company name (from SEC "registrant" field or filename), document type (10-K, 10-Q, Annual report, etc.), fiscal year (from four-digit year patterns in the first 20,000 characters), market (India/US based on currency

symbols), and thematic cluster labels (section headings extracted from the SECTION_PATTERN regex). When a Gemini API key is configured, generate_document_metadata() submits a sample of the document to the Gemini model in JSON output mode to refine the metadata.

At the chunk level, heuristic_chunk_metadata() assigns parent cluster labels (matching chunk text against document clusters), entity lists (companies, fiscal years, amounts), and placeholders for answered_questions and retrieval_nuggets. When enrichment is triggered, batches of up to 40 chunks are submitted to the Gemini model in a single JSON call, which fills in the answered_questions and retrieval_nuggets fields. These fields are prepended to the chunk text before embedding, substantially improving retrieval precision for specific queries.

8.5 Embedding Generation

Embeddings are generated using the Gemini Embedding API (model: gemini-embedding-001, output_dimensionality: 768). For document indexing, task_type=RETRIEVAL_DOCUMENT is used, which optimises the embedding for passage retrieval. For query embedding, task_type=RETRIEVAL_QUERY is used, which optimises for matching queries against indexed passages. Texts are embedded in batches of up to 40 items per API call, with a 4-second minimum interval between batches to respect the free-tier rate limit. On success, each chunk's embedding vector (a list of 768 floats) is stored in MongoDB alongside the chunk record.

8.6 Dense Retrieval and Sparse Retrieval Methods

Dense Retrieval: At query time, all indexed chunk embeddings for the requesting user are loaded from MongoDB into a numpy matrix (N x 768). The query is embedded with task_type=RETRIEVAL_QUERY. Cosine similarity is computed as the dot product of L2-normalised vectors. Chunks are ranked by descending similarity score.

Sparse Retrieval: A SparseIndex is built from all chunk texts using TfidfVectorizer (bigrams, sublinear TF, max 50,000 features) and a custom BM25 scorer. The BM25 scorer computes BM25 scores (k1=1.5, b=0.75) using per-document term frequency statistics. The TF-IDF score and BM25 score for each query are computed independently, and the two ranked lists are fused via RRF before being combined with the dense list.

8.7 Ranking and Context Selection

Reciprocal Rank Fusion (RRF) combines multiple ranked lists into a single ranking. Each document d receives a fused score: RRF(d) = sum_i w_i / (k + rank_i(d)), where rank_i(d) is d's position in the i-th list, k=60 (a smoothing constant), and w_i is a per-list weight (1.0 for BM25, 1.0 for TF-IDF, 1.0 for dense). The top-k chunks from the fused list are selected as the retrieval context. The default k is 8 for answer generation; evaluation uses k=1, 3, 5, 10, and 50.

8.8 Prompt Construction

The answer-generation prompt is structured as: (1) a system instruction telling the model to answer only from the provided context, cite sources using [SOURCE N] markers, and acknowledge when information is not present; (2) the retrieved context, formatted as numbered SOURCE blocks with document name, section, and page; (3) the user question. The prompt is passed to generate_text() or generate_text_stream() with temperature=0 for factual QA tasks (low temperature reduces hallucination).

8.9 LLM-Based Answer Generation

Text generation uses the Google Gemini API via the google-genai Python SDK. The default model (gemini-flash-lite-latest) balances speed and quality for typical financial QA workloads. The streaming endpoint (POST /api/rag/ask/stream) uses server-sent events (SSE) to deliver partial responses to the browser incrementally, providing a responsive typing effect. Non-streaming (POST /api/rag/ask) returns the full answer and sources array in a single JSON response. Both endpoints support conversation history: prior turns are included in the context window, enabling follow-up questions.

8.10 Evidence and Citation Handling

Citations are extracted from the model's response by matching [SOURCE N] markers to the corresponding entries in the context block. Each cited source is returned in the sources array of the response JSON with: document_id, document_name, section heading, page number (for PDFs), and the excerpt text. The FinRAG Copilot frontend renders these as collapsible cards below the answer, allowing the user to verify each claim against its source passage.

8.11 Transaction Feature Processing

Before scoring, transaction features are preprocessed by score_transactions(): transaction_amount is coerced to float; account_age_days is coerced to integer; kyc_verified is coerced to 0/1; channel is encoded using the saved LabelEncoder (unknown channels are mapped to -1, then clamped to a known value to avoid out-of-vocabulary errors). The four-feature vector is passed to model.predict_proba() for the fraud probability. For anomaly scoring, transaction_amount and account_age_days are log1p-transformed before being passed to the IsolationForest scorer.

8.12 Risk Prediction and Anomaly Detection

The risk_reasons list for each transaction is assembled by checking: (1) whether fraud_probability exceeds 0.7 (high model confidence); (2) whether anomaly_score exceeds 0.7 (highly unusual pattern); (3) whether transaction_amount exceeds the 95th percentile observed in training data; (4) whether account_age_days is below 30; (5) whether kyc_verified is 0. Each triggered condition adds a human-readable string to risk_reasons.

[FIGURE 8.2: Transaction Risk Scoring Pipeline -- feature extraction, RF scoring, anomaly scoring, hybrid fusion, risk level assignment]

8.13 Limitations of the Implemented Methods

1. In-memory retrieval: The dense index is rebuilt in memory for each query by loading all chunk embeddings from MongoDB. This is suitable for small to medium document collections but would become a bottleneck for large collections. A dedicated vector database (FAISS, Pinecone, Qdrant) would be required for production scale.
2. Single-user isolation: Each user's sparse index is rebuilt from scratch per query, which avoids cross-user contamination but increases per-query latency for large personal document collections.
3. ML model retraining: The fraud detection model is retrained on demand using only the current user's labelled transactions. For users with few labelled transactions, the model may have insufficient training data and will produce unreliable scores.
4. Evaluation scope: The FinDER retrieval evaluation was conducted on 25 random queries, which provides a useful benchmark result but is not statistically conclusive for all query types.

CHAPTER 9

TESTING AND PERFORMANCE EVALUATION

9.1 Testing Strategy

The testing strategy for FinRAG AI combines automated unit and integration tests (pytest) with manual functional testing of each web interface module. The test suite in the tests/ directory covers the RAG pipeline, transaction intelligence, upload pipeline, OTP flow, financial integrity checks, and security hardening. Formal performance metrics are available for the retrieval component from the FinDER benchmark evaluation. Transaction model metrics are computed during training on the held-out test split.

9.2 Functional Testing

Functional testing verifies that each feature behaves correctly given valid inputs. The key functional test cases are:

TC ID	Module	Test Case Description	Expected Result

TC01	Auth	Register with valid email and 6-digit PIN	OTP sent; account created
TC02	Auth	Register with non-numeric PIN	HTTP 400: PIN must be digits
TC03	Auth	Login with correct credentials	JWT token returned
TC04	Auth	Login with wrong PIN	HTTP 401: Invalid credentials
TC05	Upload	Upload valid CSV with all required columns	Column mapping proposal returned
TC06	Upload	Upload CSV with missing amount column	Mapping interface requests assignment
TC07	Upload	Cancel upload mid-way	AbortError: Upload cancelled toast shown
TC08	ML	Retrain model with labelled transactions	Model saved; metrics returned
TC09	ML	Score transaction before model is trained	HTTP 200: status=not_ready
TC10	RAG	Upload PDF document	Document indexed; chunk count displayed
TC11	RAG	Ask question about indexed document	Answer with source citations returned
TC12	RAG	Ask question with no documents uploaded	Response: no context available
TC13	Fraud Alerts	Raise dispute on transaction	Transaction marked as disputed
TC14	Investments	Add investment holding	Holding appears in portfolio list
TC15	Loans	Calculate EMI without entering income	Placeholder shown; no EMI computed

9.3 API Testing

API testing was performed manually using Postman and the browser DevTools Network tab. All authenticated endpoints were tested with: (a) a valid JWT token (expect 200/correct response); (b) an expired JWT token (expect 401); (c) a missing Authorization header (expect 401); (d) an invalid JSON body (expect 400). Rate-limiting behaviour was verified by sending rapid successive requests to the /login endpoint and confirming that HTTP 429 responses are returned after the configured limit.

9.4 Integration Testing

Integration testing verified the end-to-end flow from file upload to transaction scoring. The test_upload_pipeline.py file tests: file parsing for CSV, Excel, and PDF formats; column mapping; normalisation; and model scoring. The test_finrag_pipeline.py file tests: document parsing for each supported format; chunk assembly; metadata generation; embedding; and retrieval. These tests run against an isolated test MongoDB database configured via the MONGO_TEST_URI environment variable.

9.5 Retrieval and Answer Quality Evaluation

Retrieval quality is evaluated using the FinDER benchmark: 25 random queries (random seed 7) against 630 expert-annotated evidence passages from SEC 10-K filings. A hit is defined as the exact annotated passage appearing in the retrieved list at rank k. Results from the evaluation/results/finder_latest.md file:

Config	N	Recall@1	Recall@10	MRR	P50 (ms)	P95 (ms)
Sparse (TF-IDF+BM25)	25	36.0%	64.0%	0.462	10	15
Dense (Gemini emb-001)	25	72.0%	88.0%	0.778	499	686
Hybrid	25	60.0%	72.0%	0.663	522	870

(Sparse+Dense)						
Full pipeline	25	64.0%	88.0%	0.713	3589	5288

Dense retrieval achieves the best Recall@1 (72%) and MRR (0.778), indicating that Gemini embeddings are highly effective at matching queries to the exact annotated passages. Sparse retrieval is substantially faster (P50 10 ms vs 499 ms) but has lower recall. The full pipeline (including metadata-enriched chunks) matches dense Recall@10 at 88% with improved MRR (0.713) relative to the bare hybrid configuration, demonstrating the benefit of metadata enrichment.

9.6 Transaction Risk Evaluation

Transaction model metrics are computed during training on the held-out test split (80/20 stratified split, random_state=42). Metrics reported include: ROC-AUC, PR-AUC, fraud-class Precision, Recall, F1-score, false-positive rate, and confusion matrix. The exact metric values depend on the user's labelled transaction data and vary per training run. During development with the sample CSV (approximately 1,083 transactions, ~5.8% fraud rate), the RandomForest component achieved ROC-AUC > 0.90 and the hybrid risk score reduced the false-positive rate relative to the classifier alone.

9.7 Test Results Summary

The automated test suite (pytest tests/) passed all tests related to: document parsing (all four formats), chunk assembly, metadata heuristics, embedding batch logic, sparse index construction, RRF fusion, upload pipeline, OTP generation and verification, and JWT token validation. Tests for end-to-end RAG answer quality require a live Gemini API key and are tagged as integration tests to be run separately.

9.8 Discussion

The retrieval results confirm that dense retrieval with Gemini embeddings significantly outperforms sparse retrieval on the FinDER benchmark for financial document passages, consistent with findings in the RAG literature. The hybrid configuration does not improve over dense alone in this evaluation, suggesting that the sparse component may add noise on a benchmark where questions are semantically rich. However, sparse retrieval remains valuable for exact-match queries involving specific numerical values or company names. The full pipeline, which

incorporates chunk-level metadata enrichment, matches the dense Recall@10 of 88% while improving MRR, suggesting that metadata grounding adds complementary signal beyond the passage text alone.

CHAPTER 10

RESULTS AND WEBSITE SCREENSHOTS

10.1 Overview of Results

This chapter presents the operational results of the FinRAG AI system through descriptions of its key interface screens. The system delivers a fully functional web application with twelve modules, all accessible through a consistent sidebar navigation and available after authentication.

10.2 Login / Authentication Page

The authentication page features a two-panel layout: a gradient-background left panel displaying four trust badges (ML fraud detection, FinRAG Copilot, 6-digit PIN + OTP security, data privacy), and a right panel with tabbed login and registration forms. The login form accepts email and 6-digit PIN with a show/hide toggle. The registration form includes the same PIN field and validates the PIN as exactly 6 numeric digits client-side. The forgot-password link initiates an OTP-based password reset flow inline on the same page.

[FIGURE 10.1: Authentication Page -- Two-panel login/register interface with trust badges and OTP verification form]

10.3 Dashboard

The dashboard greets the logged-in user by name and presents five metric cards at the top (total transactions, model-flagged transactions, disputed transactions, security status, net cash flow). Below the cards, a bento-grid layout hosts six charts: transaction volume bar chart, trust score half-doughnut gauge, income vs. expense grouped bar chart, spending categories doughnut with colour-coded legend, fraud alerts trend line chart, and spending forecast line chart with a dashed AI prediction extension. A recurring payments detector and recent alerts list complete the dashboard.

[FIGURE 10.2: Main Dashboard -- Bento-grid showing metric cards, six charts, and recent alerts]

10.4 Upload Interface

The upload page provides a drag-and-drop zone for transaction files (CSV, Excel, PDF, image). A progress bar shows upload and processing progress. A Cancel Upload button (using AbortController) allows the user to abort mid-upload. After upload, a column-mapping step presents a table matching detected columns to the canonical schema. An upload history section lists past import batches with transaction counts and rollback capability.

[FIGURE 10.3: Upload Transaction Data Interface -- Drag-and-drop zone, column mapping table, and upload history]

10.5 Fraud Alerts

The fraud alerts page lists all model-flagged transactions as cards with severity-coloured borders and badges (Critical in dark red, High in red, Medium in amber, Low in grey). Each card shows the transaction ID, amount, channel, customer, and timestamp. An "Ask AI" button triggers a Gemini-powered fraud analysis that explains why the transaction is suspicious. Skeleton loaders display during the initial data fetch.

[FIGURE 10.4: Fraud Alerts -- Severity-coloured cards with Critical/High/Medium/Low badges and Ask AI button]

10.6 FinRAG Copilot Interface

The FinRAG Copilot is a three-panel interface: a left panel listing uploaded documents with file type icons and chunk counts; a central panel with a question input field, conversation history, and streamed answer display; and a right panel showing the cited source passages. Each cited passage is displayed as a card with the document name, section heading, page number, and excerpt. The conversation history is preserved within the session and can be retrieved via GET /api/rag/conversations.

[FIGURE 10.5: FinRAG Copilot -- Document library, question/answer interface, and source citation cards]

10.7 Loan EMI Calculator

The EMI calculator presents three sliders (loan amount: INR 10,000--1,00,00,000; interest rate: 0--30%; tenure: 1--30 years) that update display labels in real time. A monthly income input field and "Estimate affordability" button trigger the EMI calculation and affordability check. The result panel shows the computed monthly

EMI, total interest, and total payment. The eligibility result section indicates whether the EMI is within the 40% of income affordability guideline.

[FIGURE 10.6: Loan EMI Calculator -- Sliders, income field, and EMI result with affordability indicator]

10.8 Investment Portfolio

The investment portfolio page shows a demo portfolio banner, total portfolio value, a holdings list with asset name, type, quantity, purchase price, current price, and P&L, a doughnut chart of allocation by asset type, and an "AI Portfolio Analysis" button that submits the portfolio to the Gemini API for an allocation recommendation. Users can add new holdings via a form and remove existing holdings via a styled confirm modal.

[FIGURE 10.7: Credit Health -- Score gauge and factor breakdown cards]

[FIGURE 10.8: Investment Portfolio -- Holdings list, allocation chart, and AI analysis result]

10.9 Discussion of Observed Results

The FinRAG AI system successfully delivers all ten planned modules with a consistent UI, fully functional backend APIs, and verified ML and RAG pipelines. The FinRAG Copilot produces coherent, cited answers to financial questions about uploaded documents, with dense retrieval achieving Recall@10 of 88% on the FinDER benchmark. The transaction intelligence module produces interpretable risk scores with human-readable explanations. All alert() and confirm() calls have been replaced with the custom toast/modal system, providing a polished and consistent user experience.

CHAPTER 11

CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

This project successfully developed FinRAG AI -- a unified AI framework for financial risk and retrieval-augmented intelligence. The system integrates two complementary intelligence engines within a single Flask web application backed by MongoDB: a hybrid transaction risk engine (RandomForestClassifier + IsolationForest, hybrid risk score with human-readable explanations) and a financial document Q&A copilot (metadata-enriched chunking, TF-IDF/BM25 sparse + Gemini dense hybrid retrieval, Gemini generative answer production with cited sources).

The platform delivers twelve functional modules covering authentication (email + 6-digit PIN + OTP), transaction management, fraud alert review with severity classification, predictive analytics, credit health scoring, investment portfolio management, loan EMI calculation, AI model insights, and the FinRAG Copilot. All modules are accessible through a consistent, responsive dark-themed interface with Chart.js-powered dashboards, a global toast notification system, and skeleton loading states.

Retrieval quality was evaluated on the FinDER benchmark, where dense retrieval (gemini-embedding-001) achieved Recall@10 of 88% and MRR of 0.778 on a 25-query evaluation, demonstrating that Gemini embeddings are highly effective for financial passage retrieval. The transaction intelligence module was verified to produce calibrated risk scores and interpretable risk reasons on the supplied sample transaction dataset.

11.2 Achievement of Project Objectives

All seven project objectives stated in Chapter 1 were achieved:

Objective 1 -- Hybrid risk scoring pipeline: Implemented and verified. RandomForestClassifier + IsolationForest with F1-optimal threshold selection, producing risk_score, risk_level, and risk_reasons for each transaction.

Objective 2 -- Document ingestion pipeline: Implemented for PDF, HTML, Markdown, and TXT. Structure-preserving parsing with heading detection and table preservation. Heuristic and LLM-enriched metadata generation at document and chunk level.

Objective 3 -- Dual-mode retrieval and FinDER evaluation: Implemented. TF-IDF+BM25 sparse and Gemini dense retrieval fused via RRF. FinDER evaluation completed with dense Recall@10=88%, MRR=0.778.

Objective 4 -- Gemini generative integration for grounded answers: Implemented via POST /api/rag/ask and /api/rag/ask/stream. Every answer includes SOURCE citations mapped back to the retrieved passages.

Objective 5 -- Multi-module Flask + MongoDB web application: Implemented. Flask 3.x, MongoDB, JWT+bcrypt+OTP auth, Chart.js dashboards, twelve modules.

Objective 6 -- End-to-end evaluation: Completed. Transaction model metrics computed on training split; retrieval metrics from FinDER benchmark.

Objective 7 -- Production-grade deployment features: Implemented. Flask-Limiter, Flask-Caching, Flask-Compress, gunicorn WSGI server, structured logging.

11.3 Current Limitations

1. In-memory retrieval does not scale to very large document collections (thousands of documents) without a dedicated vector database.
2. The transaction ML model requires the user to provide labelled data (fraud = 1) for effective training; without labels, risk scoring defaults to anomaly-only detection.
3. The FinDER evaluation used a small 25-query sample; full-scale evaluation is needed for statistical confidence.
4. Answer quality (faithfulness, hallucination rate) was not formally measured with a judge model.
5. The system is designed for single-user academic use and lacks multi-tenant isolation, data retention policies, and GDPR compliance workflows required for institutional deployment.

11.4 Future Enhancements

1. Vector Database Integration: Replace in-memory cosine similarity with a dedicated vector database (FAISS, Qdrant, or Pinecone) to support large document collections

with sub-second dense retrieval latency.

2. Cross-Modal Retrieval: Extend document parsing to support DOCX and PowerPoint files, enabling retrieval from presentations and structured reports.

3. Real-Time Transaction Feeds: Integrate with open banking APIs (e.g., Account Aggregator framework in India) to enable real-time transaction monitoring rather than batch import.

4. Multi-User and Team Features: Add role-based access control and shared document libraries to support team-level financial analysis workflows.

5. Mobile Application: Develop a React Native or Flutter mobile client consuming the existing REST API to provide on-the-go transaction monitoring and document Q&A.

6. Advanced Reranking: Integrate a cross-encoder reranker (e.g., a fine-tuned BERT model) between the retriever and the generative model to further improve answer precision.

7. Formal Answer Quality Evaluation: Implement an LLM-as-judge evaluation pipeline to measure faithfulness, answer relevance, and hallucination rate on a curated financial QA test set.

8. Explainable AI Dashboard: Add SHAP-based feature importance visualisations for individual transaction risk scores, providing deeper interpretability for fraud analysts.

REFERENCES

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS 2020), 33, 9459-9474.

[2] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM), 413-422.

[3] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[4] Google. (2024). Gemini API Documentation. Google for Developers. https://ai.google.dev/docs

[5] Flask Documentation. (2024). Flask 3.x -- A lightweight WSGI web application framework. Pallets Projects. https://flask.palletsprojects.com/

[6] MongoDB Documentation. (2024). MongoDB Manual. MongoDB Inc. https://www.mongodb.com/docs/manual/

[7] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of EMNLP-IJCNLP 2019.

[8] PyMongo Documentation. (2024). PyMongo 4.x API Reference. MongoDB Inc. https://pymongo.readthedocs.io/

[9] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In Proceedings of SIGIR 2009, 758-759.

[10] PyJWT Documentation. (2024). PyJWT -- JSON Web Tokens for Python. https://pyjwt.readthedocs.io/

[11] pypdf Documentation. (2024). pypdf -- A pure-Python PDF library. https://pypdf.readthedocs.io/

[12] Chart.js Documentation. (2024). Chart.js -- Simple yet flexible JavaScript charting. https://www.chartjs.org/docs/

[13] scikit-learn Documentation. (2024). sklearn.ensemble.RandomForestClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[14] scikit-learn Documentation. (2024). sklearn.ensemble.IsolationForest. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

APPENDIX A

API ENDPOINT REFERENCE

Method	Endpoint	Auth	Description

POST	/register	No	Register a new user (name, email, PIN)
POST	/login	No	Authenticate and receive JWT token
POST	/verify-registration	No	Verify email via OTP
POST	/resend-registration-otp	No	Resend OTP for registration
POST	/forgot-password	No	Send password reset OTP
POST	/reset-password	No	Reset password with OTP and new PIN
GET	/api/dashboard	Yes	Aggregated dashboard data (stats, charts)
POST	/upload	Yes	Upload transaction file; returns column mapping
POST	/upload/confirm	Yes	Confirm column mapping and import transactions
POST	/upload/rollback	Yes	Roll back an import batch
GET	/api/transactions	Yes	List transactions (paginated, filtered)
DELETE	/api/transactions/{id}	Yes	Delete a transaction
POST	/api/transactions/{id}/dispute	Yes	Raise a dispute on a transaction
GET	/api/alerts	Yes	List model-flagged transactions

POST	/api/retrain	Yes	Trigger ML model retraining
POST	/api/predict	Yes	Score a single hypothetical transaction
POST	/api/rag/index	Yes	Index (embed) a document's chunks
POST	/api/rag/ask	Yes	Ask a question (non-streaming response)
POST	/api/rag/ask/stream	Yes	Ask a question (SSE streaming response)
GET	/api/rag/documents	Yes	List uploaded documents
DELETE	/api/rag/documents/{id}	Yes	Delete a document and its chunks
POST	/api/rag/ingest/sec	Yes	Ingest a SEC EDGAR filing
POST	/api/rag/ingest/url	Yes	Ingest a document from URL
GET	/api/investments	Yes	List investment holdings
POST	/api/investments	Yes	Add a new investment holding
POST	/api/check-loan-eligibility	Yes	Check loan affordability
GET	/api/credit-score	Yes	Get credit health score
GET	/api/profile	Yes	Get user profile
PUT	/api/profile	Yes	Update user profile
POST	/api/profile/password	Yes	Change 6-digit PIN

APPENDIX B

SAMPLE TRANSACTION CSV FORMAT

The system accepts CSV files with the following canonical column schema. Column names are case-insensitive; the column-mapping interface handles alternate header spellings from different bank export formats.

Column Name	Data Type	Example Value	Required
transaction_id	String	TXN20240415001	No (auto-generated if absent)
customer_id	String	CUST1234	No
transaction_amount	Float	15000.00	Yes
transaction_date	String (ISO 8601)	2024-04-15T14:32:00	Yes
channel	String	UPI	Yes
category	String	Groceries	No
description	String	SWIGGY ORDER 8823	No
is_fraud	Integer (0/1)	0	No (required for model training)
account_age_days	Integer	365	No (defaults to 0)

kyc_verified	Integer (0/1)	1	No (defaults to 0)

APPENDIX C

SETUP AND INSTALLATION INSTRUCTIONS

C.1 Prerequisites

Python 3.10 or later, Git, MongoDB Atlas account (free tier), Google Gemini API key (free tier), SMTP-capable email account for OTP delivery.

C.2 Installation Steps

1. Clone the repository: git clone https://github.com/JitendraSri/BFSI

2. Create a virtual environment: python -m venv .venv && .venv\Scripts\activate (Windows) or source .venv/bin/activate (Linux/macOS)

3. Install dependencies: pip install -r requirements.txt

4. Create a .env file in the project root with the following variables:

MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/<dbname>

JWT_SECRET=<your-random-secret-key>

GEMINI_API_KEY=<your-gemini-api-key>

SMTP_HOST=smtp.gmail.com

SMTP_PORT=587

SMTP_USER=<your-email@gmail.com>

SMTP_PASS=<your-app-password>

SENDER_EMAIL=<your-email@gmail.com>

C.3 Running the Application

Development server: python app.py (runs on http://127.0.0.1:5000)

Production server: gunicorn -w 4 -b 0.0.0.0:5000 app:app

C.4 First Use

Navigate to http://localhost:5000, register an account, verify via OTP, and log in. Upload the sample CSV from static/samples/sample_transactions.csv to populate

initial transaction data. Click "Retrain Model" in Predictive Analytics to train the fraud detection model. Upload a PDF annual report in FinRAG Copilot and ask questions.